Python Code

```python
import hashlib
import html
import json
from collections import Counter
from pathlib import Path

import numpy as np
import pandas as pd
import plotly.express as px
import plotly.graph_objects as go
import streamlit as st

st.set_page_config(page_title="KelpNLP Dataset Explorer", page_icon="🗂️", layout="wide", initial_sidebar_state="expanded")

ROOT = Path(__file__).resolve().parent.parent
DATASET_DIR = ROOT / "dataset"
SUPPORT_DIR = DATASET_DIR / "pendukung"
GOLD_PATH = DATASET_DIR / "KelpNLP_dataset_annotation.jsonl"
CLEAN_PATH = DATASET_DIR / "KelpNLP_dataset_clean.csv"
SHA_GOLD = "8a8a77faa4fb7aa76b6d3d178cbc62fa4138525411bd80ab3971910ec6138b3d"

LABELS = ["INSULTING", "ACCUSING", "MOCKING", "PROVOKING", "CURSING", "NONE"]
ATT_LABELS = LABELS[:5]
ENT_TYPES = ["GOV_ORG", "NONGOV_ORG", "PERSON", "GROUP"]

# definisi = guideline final (KelpNLP_annotation_guideline_final, bagian 3.2 dan 3.7)
LABEL_DESC = {
    "INSULTING": "Menyatakan, langsung atau lewat ungkapan baku, bahwa diri target bodoh, tidak berguna, "
                 "tidak becus, atau buruk sifatnya.",
    "ACCUSING": "Menyatakan atau menyiratkan bahwa target berbuat tercela, punya motif buruk, berpihak "
                "tersembunyi, atau lalai dan pilih kasih.",
    "MOCKING": "Membuat target tampak konyol lewat cara tidak harfiah: ironi, pujian palsu, tawa ejekan, "
               "pelesetan, tantangan sinis, julukan ejekan.",
    "PROVOKING": "Mengajak atau menuntut tindakan yang menghukum, menyingkirkan, atau memusuhi target "
                 "(bubarkan, tutup, ganti, mundur, boikot).",
    "CURSING": "Mendoakan, mengharapkan, atau menyoraki kemalangan target, tanpa meminta pihak tertentu bertindak.",
    "NONE": "Tidak memuat satu pun dari lima jenis serangan, termasuk kritik, saran, dan pembelaan.",
}
ENT_DESC = {
    "GOV_ORG": "Lembaga negara atau pemerintah, termasuk sebutan [pemerintah] dan BUMN. "
               "\"negara\", \"Indonesia\", dan \"bangsa\" tidak disorot.",
```

```
    "NONGOV_ORG": "Organisasi bukan pemerintah yang disebut namanya: perusahaan dan merek, yayasan, klub, "
                  "federasi, LSM, dan kelompok bernama (mis. Taliban).",
    "PERSON": "Nama orang, termasuk sapaan yang menempel ([Pak Jokowi]). Tokoh fiksi dan julukan tidak disorot.",
    "GROUP": "Sebutan umum untuk sekumpulan atau kategori manusia: bukan nama organisasi, bukan satu individu, "
             "bukan kata ganti, dan bukan jabatan (komisioner, pejabat tidak disorot).",
}
LABEL_COLORS = {
    "INSULTING": "#e34948",
    "ACCUSING": "#7c4dbd",
    "MOCKING": "#eda100",
    "PROVOKING": "#2a78d6",
    "CURSING": "#1baf7a",
    "NONE": "#898781",
}
ENTITY_COLORS = {"GOV_ORG": "#2a78d6", "NONGOV_ORG": "#eb6834", "PERSON": "#1baf7a", "GROUP": "#eda100"}
JENIS_NORM = {
    "kamus": "#2a78d6",
    "reduplikasi": "#eb6834",
    "tawa": "#1baf7a",
    "huruf_berulang": "#eda100",
    "huruf_berulang+kamus": "#7c4dbd",
}
WARNA = {"utama": "#2a78d6", "teks": "#0b0b0b", "teks2": "#52514e", "grid": "#e1e0d9", "latar": "#fcfcfb"}
BERITA = {
    1: "KPAI Survei Ke Anak Yang Mana??",
    2: "#bubarkanKPAI Trending di Twitter",
    3: "PB Djarum Hentikan Audisi Bulutangkis",
}
IAA = {"fleiss": 0.840, "f1_ner": 0.926}


@st.cache_data(show_spinner=False)
def load_gold():
    with open(GOLD_PATH, encoding="utf-8") as fh:
        return [json.loads(line) for line in fh if line.strip()]


@st.cache_data(show_spinner=False)
def load_clean():
    df = pd.read_csv(CLEAN_PATH, encoding="utf-8-sig", keep_default_na=False, dtype={"id": str})
    df["in_domain"] = df["in_domain"].astype(bool)
    return df


@st.cache_data(show_spinner=False)
def load_support(name):
```

```
    return pd.read_csv(SUPPORT_DIR / name, encoding="utf-8-sig", keep_default_na=False)


@st.cache_data(show_spinner=False)
def gold_map():
    return {g["id"]: g for g in load_gold()}


def span_token_len(bio):
    out = []
    for t in bio:
        if t.startswith("B-"):
            out.append(1)
        elif t.startswith("I-"):
            out[-1] += 1
    return out


@st.cache_data(show_spinner=False)
def build_tables():
    gold = load_gold()
    rows, spans = [], []
    for g in gold:
        adj = g["adjudication"]
        rows.append({
            "id": g["id"],
            "thread": g["meta"]["thread"],
            "berita": g["meta"]["berita"],
            "text": g["text"],
            "labels": tuple(g["labels"]),
            "n_label": len(g["labels"]),
            "n_char": len(g["text"]),
            "n_kata": len(g["text"].split()),
            "n_token": len(g["tokens"]),
            "n_entitas": len(g["spans"]),
            "tipe_entitas": tuple(sorted({s["label"] for s in g["spans"]})),
            "adj_label": adj["label"],
            "adj_ner": adj["ner"],
            "manual": adj.get("manual") or "",
            "status_manual": status_adjudikasi(adj.get("manual") or ""),
        })
        for s, nt in zip(g["spans"], span_token_len(g["ner_bio"])):
            spans.append({"id": g["id"], "tipe": s["label"], "teks": s["text"], "n_char": s["end"] - s["start"], "n_token": nt})
    df = pd.DataFrame(rows).set_index("id")
    Y = pd.DataFrame({l: [int(l in g["labels"]) for g in gold] for l in LABELS}, index=[g["id"] for g in gold])
    sp = pd.DataFrame(spans)
    tag = Counter()
    for g in gold:
        tag.update("O" if t == "O" else t[:2] for t in g["ner_bio"])
    return df, Y, sp, tag

```

```python

def active_threads():
    return st.session_state.get("thread_filter") or []


def fig_base(fig, height=380):
    fig.update_layout(
        template="plotly_white",
        height=height,
        margin=dict(l=10, r=30, t=46, b=10),
        font=dict(size=13, color=WARNA["teks"]),
        title_font=dict(size=15),
        title_x=0,
    )
    return fig


# karakter HTML dan markdown diganti entity dalam SATU kali lewat (str.translate),
# supaya teks komentar tampil apa adanya (mis. "b*d*h" tidak jadi huruf miring)
_ESC = str.maketrans({
    "&": "&amp;", "<": "&lt;", ">": "&gt;", '"': "&quot;", "'": "&#39;",
    "*": "&#42;", "_": "&#95;", "~": "&#126;", "`": "&#96;", "$": "&#36;",
    "#": "&#35;", "[": "&#91;", "]": "&#93;",
})


def esc(teks):
    return str(teks).translate(_ESC)


def status_adjudikasi(manual):
    # pisahkan keputusan manual (18 teks) dari koreksi pedoman temuan T3–T7 (7 teks)
    if not manual:
        return ""
    return "koreksi pedoman" if manual.startswith("koreksi pedoman") else "keputusan manual"


def chip(label, warna=None):
    c = warna or LABEL_COLORS.get(label, "#888888")
    return (
        f'<span style="background:{c}1a;color:{c};border:1px solid {c};border-radius:12px;'
        f'padding:2px 10px;font-weight:600;font-size:12px;white-space:nowrap">{label}</span>'
    )


def label_chips(labels):
    return " ".join(chip(l) for l in labels)


def entity_legend(types):
    return " ".join(
```

```
        f'<span style="background:{ENTITY_COLORS[t]}26;color:{ENTITY_COLORS[t]};border:1px solid {ENTITY_COLORS[t]};'
        f'border-radius:10px;padding:1px 8px;font-size:12px;font-weight:600">■ {t}</span>'
        for t in types
    )


def highlight_spans(text, spans, emphasize=None):
    parts, pos = [], 0
    for s in sorted(spans, key=lambda x: x["start"]):
        if s["start"] < pos:
            continue
        parts.append(esc(text[pos:s["start"]]))
        c = ENTITY_COLORS.get(s["label"], "#888888")
        teks = esc(s["text"])
        sub = f'<sub style="font-size:10px;color:{c}">{s["label"]}</sub>'
        if emphasize and s["text"].lower() == emphasize.lower():
            parts.append(
                f'<span style="background:{c}2e;border:2px solid {c};border-radius:4px;padding:1px 5px;'
                f'font-weight:700">{teks}{sub}</span>'
            )
        else:
            parts.append(
                f'<span style="background:{c}26;border-bottom:3px solid {c};border-radius:3px;'
                f'padding:1px 3px">{teks}{sub}</span>'
            )
        pos = s["end"]
    parts.append(esc(text[pos:]))
    return "".join(parts)


def render_detail(gid, clean_idx):
    g = gold_map()[gid]
    st.markdown(
        f"**{gid}** · Utas {g['meta']['thread']} — {g['meta']['berita']} · "
        f"label asli dataset: `{clean_idx.at[gid, 'orig_label_name']}`"
    )
    st.markdown(label_chips(g["labels"]), unsafe_allow_html=True)
    st.markdown(" ", unsafe_allow_html=True)
    st.markdown(highlight_spans(g["text"], g["spans"]), unsafe_allow_html=True)
    if g["spans"]:
        st.markdown(" ", unsafe_allow_html=True)
        st.markdown(entity_legend(sorted({s["label"] for s in g["spans"]})), unsafe_allow_html=True)
    ann_rows = []
    for a, v in g["annotations"].items():
        ann_rows.append({
            "annotator": a,
            "label": " + ".join(v["labels"]) or "—",
            "entitas": " ".join(f'[{s["text"]}] {s["label"]}' for s in v["spans"]) or "—",
```

```
238             })
239         st.dataframe(pd.DataFrame(ann_rows), width="stretch", hide_index=True)
240         adj = g["adjudication"]
241         pesan = f'Adjudikasi — label: **{adj["label"]}** · NER: **{adj["ner"]}**'
242         if adj.get("manual"):
243             pesan += f" · {status_adjudikasi(adj['manual'])}: {adj['manual']}"
244         st.info(pesan)
245         with st.expander("text_raw & text_norm"):
246             st.markdown(f"**text_raw**\n\n```\n{clean_idx.at[gid, 'text_raw']}\n```")
247             st.markdown(f"**text_norm**\n\n```\n{clean_idx.at[gid, 'text_norm']}\n```")
248 
249 
250 def page_overview():
251     st.title("📊 Overview")
252     st.caption("Gold dataset anotasi multilabel & NER — komentar polemik KPAI vs PB
    Djarum (2019)")
253     thr = active_threads()
254     if not thr:
255         st.warning("Pilih minimal satu utas berita pada filter global di sidebar.")
256         return
257     df, Y, sp, tag = build_tables()
258     d = df[df["thread"].isin(thr)]
259     Yf = Y.loc[d.index]
260     spf = sp[sp["id"].isin(d.index)]
261 
262     k1, k2, k3, k4, k5 = st.columns(5)
263     k1.metric("Teks gold", f"{len(d):,}")
264     k2.metric("Entitas", f"{len(spf):,}")
265     k3.metric("Token", f"{d['n_token'].sum():,}")
266     k4.metric("Label per teks", f"{d['n_label'].mean():.2f}")
267     k5.metric("Teks dengan entitas", f"{(d['n_entitas'] > 0).mean():.1%}")
268     k1, k2, k3 = st.columns(3)
269     k1.metric("Fleiss' κ rata-rata (5 label)", f"{IAA['fleiss']:.2f}", "hampir
    sempurna")
270     k2.metric("NER F1 exact (3 pasangan)", f"{IAA['f1_ner']:.2f}")
271     sepakat = d["adj_label"].str.startswith("sepakat")
272     k3.metric("Label sepakat 3/3", f"{sepakat.sum():,}", f"{sepakat.mean():.0%} dari
    teks")
273 
274     st.divider()
275     clean = load_clean()
276     removed = load_support("removed_rows.csv")
277     sid = load_support("sample_ids.csv")
278     tahap = ["Dataset asli", "Setelah cleaning", "Corpus domain", "Sampel anotasi",
    "Gold dataset"]
279     nilai = [
280         len(clean) + len(removed),
281         len(clean),
282         int(clean["in_domain"].sum()),
283         int((sid["bagian"] == "utama").sum()),
284         len(load_gold()),
285     ]
286     fig = go.Figure(go.Funnel(
287         y=tahap, x=nilai, textinfo="value",
```

```
        marker=dict(color=WARNA["utama"], opacity=[1.0, 0.88, 0.76, 0.64, 0.52]),
        connector=dict(line=dict(color=WARNA["grid"], width=2)),
    ))
    fig.update_layout(title="Alur pembentukan dataset")
    st.plotly_chart(fig_base(fig, 320), width="stretch")
    st.caption("50 teks pilot dipakai untuk uji coba guideline dan tidak masuk gold dataset.")

    kiri, kanan = st.columns(2)
    dist = Yf.sum().loc[LABELS].sort_values()
    fig = go.Figure(go.Bar(
        x=dist.values, y=dist.index, orientation="h",
        marker_color=[LABEL_COLORS[l] for l in dist.index],
        text=[f"{v} ({v / len(Yf):.1%})" for v in dist.values], textposition="outside",
        hovertemplate="%{y}: %{x} teks<extra></extra>",
    ))
    fig.update_xaxes(range=[0, dist.max() * 1.28])
    fig.update_layout(title="Distribusi label gold")
    kiri.plotly_chart(fig_base(fig), width="stretch")

    ent_n = spf["tipe"].value_counts().reindex(ENT_TYPES).fillna(0).astype(int)
    urut = ent_n.sort_values()
    fig = go.Figure(go.Bar(
        x=urut.values, y=urut.index, orientation="h",
        marker_color=[ENTITY_COLORS[t] for t in urut.index],
        text=[f"{v}" for v in urut.values], textposition="outside",
        hovertemplate="%{y}: %{x} entitas<extra></extra>",
    ))
    fig.update_xaxes(range=[0, urut.max() * 1.22])
    fig.update_layout(title="Distribusi entitas NER")
    kanan.plotly_chart(fig_base(fig), width="stretch")

    per_thread = df.groupby("thread").size().reindex([1, 2, 3]).fillna(0).astype(int)
    fig = go.Figure(go.Bar(
        x=[f"Utas {t}" for t in per_thread.index], y=per_thread.values,
        marker_color=[WARNA["utama"], "#eb6834", "#1baf7a"],
        text=per_thread.values, textposition="outside",
        customdata=[BERITA[t] for t in per_thread.index],
        hovertemplate="%{customdata}<br>%{x}: %{y} teks<extra></extra>",
    ))
    fig.update_yaxes(range=[0, per_thread.max() * 1.2])
    fig.update_layout(title="Teks gold per utas berita (tanpa filter)")
    st.plotly_chart(fig_base(fig, 300), width="stretch")

    st.subheader("Taxonomy")
    tab1, tab2 = st.tabs(["🏷️ Label serangan (multilabel)", "🎯 Entitas NER"])
    with tab1:
        baris = []
        for l in LABELS:
            kandidat = df[df["labels"].map(lambda L: l in L)].sort_values("n_kata")
            teks_contoh = kandidat.iloc[0]["text"] if len(kandidat) else ""
            contoh = esc(teks_contoh[:140]) + ("…" if len(teks_contoh) > 140 else "")
            baris.append(
```

```
                f'<tr><td style="padding:7px 12px;border-bottom:1px solid {WARNA["grid"]}">{chip(l)}</td>'
                f'<td style="padding:7px 12px;border-bottom:1px solid {WARNA["grid"]}">{LABEL_DESC[l]}</td>'
                f'<td style="padding:7px 12px;border-bottom:1px solid {WARNA["grid"]};text-align:center">'
                f'<b>{Y[l].sum()}</b> ({Y[l].mean():.1%})</td>'
                f'<td style="padding:7px 12px;border-bottom:1px solid {WARNA["grid"]};color:{WARNA["teks2"]};'
                f'font-style:italic">{contoh}</td></tr>'
            )
        kepala = (
            f'<tr><th style="text-align:left;padding:7px 12px">Label</th>'
            f'<th style="text-align:left;padding:7px 12px">Definisi</th>'
            f'<th style="text-align:center;padding:7px 12px">Teks</th>'
            f'<th style="text-align:left;padding:7px 12px">Contoh teks</th></tr>'
        )
        st.markdown(
            f'<table style="width:100%;border-collapse:collapse">{kepala}{"".join(baris)}</table>',
            unsafe_allow_html=True,
        )
        st.caption("UNANNOTATABLE (teks tidak bisa dipahami/dinilai) hanya dipakai saat anotasi; "
                   "1 teks berlabel ini dikeluarkan dari gold dataset.")
    with tab2:
        baris = []
        for t in ENT_TYPES:
            top = sp[sp["tipe"] == t]["teks"].value_counts()
            contoh = ", ".join(f"<code>{esc(x)}</code>" for x in top.head(3).index)
            n_teks = df["tipe_entitas"].map(lambda T: t in T).sum()
            baris.append(
                f'<tr><td style="padding:7px 12px;border-bottom:1px solid {WARNA["grid"]}">{chip(t, ENTITY_COLORS[t])}</td>'
                f'<td style="padding:7px 12px;border-bottom:1px solid {WARNA["grid"]}">{ENT_DESC[t]}</td>'
                f'<td style="padding:7px 12px;border-bottom:1px solid {WARNA["grid"]};text-align:center">'
                f'<b>{(sp["tipe"] == t).sum()}</b> ({n_teks / len(df):.0%} teks)</td>'
                f'<td style="padding:7px 12px;border-bottom:1px solid {WARNA["grid"]}">{contoh}</td></tr>'
            )
        kepala = (
            f'<tr><th style="text-align:left;padding:7px 12px">Tipe</th>'
            f'<th style="text-align:left;padding:7px 12px">Definisi</th>'
            f'<th style="text-align:center;padding:7px 12px">Entitas</th>'
            f'<th style="text-align:left;padding:7px 12px">Contoh tersering</th></tr>'
        )
        st.markdown(
            f'<table style="width:100%;border-collapse:collapse">{kepala}{"".join(baris)}</table>',
            unsafe_allow_html=True,
        )

```

```
383
384 def page_browse():
385     st.title("🔎 Browse & Search")
386     st.caption("Cari teks, filter berdasarkan label atau entitas, lalu lihat label
    multilabel, span NER, "
387                "dan jawaban tiap annotator")
388     thr = active_threads()
389     if not thr:
390         st.warning("Pilih minimal satu utas berita pada filter global di sidebar.")
391         return
392     df, Y, sp, tag = build_tables()
393     clean_idx = load_clean().set_index("id")
394
395     fcol, dcol = st.columns([1, 3], gap="large")
396     with fcol:
397         st.subheader("Filter & pencarian")
398         q = st.text_input("Cari teks / ID", placeholder="mis. kpai, ANC-1123",
    key="cari")
399         pilih_label = st.multiselect("Label gold (semua)", LABELS)
400         pilih_ent = st.multiselect("Tipe entitas (semua)", ENT_TYPES)
401         q_ent = st.text_input("Teks entitas", placeholder="mis. pb djarum, anak",
    key="cari_entitas",
402                               help="Teks yang punya entitas gold berisi kata ini
    (huruf besar/kecil diabaikan)")
403         status = st.selectbox(
404             "Status adjudikasi",
405             ["Semua", "sepakat 3/3", "mayoritas 2/3", "keputusan manual", "koreksi
    pedoman"],
406             help="keputusan manual = 18 teks yang diputuskan tim; koreksi pedoman = 7
    teks yang span-nya "
407                  "dikoreksi agar sesuai guideline (temuan T3–T7)",
408         )
409         orig = st.selectbox("Label asli dataset", ["Semua", "not_abusive",
    "abusive_not_offensive", "abusive_offensive"])
410         n_tampil = st.number_input("Baris ditampilkan", 10, 200, 25, 10)
411
412     m = pd.Series(True, index=df.index)
413     m &= df["thread"].isin(thr)
414     if q:
415         m &= df["text"].str.contains(q, case=False, regex=False) |
    df.index.str.contains(q, case=False, regex=False)
416     if pilih_label:
417         m &= df["labels"].map(lambda L: all(l in L for l in pilih_label))
418     if pilih_ent:
419         m &= df["tipe_entitas"].map(lambda T: all(e in T for e in pilih_ent))
420     if q_ent:
421         id_ent = set(sp.loc[sp["teks"].str.contains(q_ent, case=False, regex=False),
    "id"])
422         m &= df.index.isin(id_ent)
423     if status == "sepakat 3/3":
424         m &= df["adj_label"].str.startswith("sepakat")
425     elif status == "mayoritas 2/3":
426         m &= df["adj_label"].str.startswith("mayoritas")
427     elif status in ("keputusan manual", "koreksi pedoman"):
```

```
428         m &= df["status_manual"] == status
429     if orig != "Semua":
430         m &= df.index.map(lambda i: clean_idx.at[i, "orig_label_name"]) == orig
431 
432     tampil = df.loc[m]
433     with dcol:
434         st.caption(f"**{len(tampil):,}** teks cocok")
435         show = tampil.head(n_tampil).reset_index()
436         show["labels"] = show["labels"].map(" + ".join)
437         st.dataframe(
438             show[["id", "text", "labels", "n_entitas", "adj_label"]],
439             width="stretch", height=330, hide_index=True,
440             column_config={
441                 "text": st.column_config.TextColumn("teks", width="large"),
442                 "labels": st.column_config.TextColumn("label gold", width="medium"),
443                 "n_entitas": st.column_config.NumberColumn("entitas", format="%d"),
444                 "adj_label": st.column_config.TextColumn("adjudikasi",
width="medium"),
445             },
446         )
447 
448     if not len(tampil):
449         st.info("Tidak ada teks yang cocok dengan filter.")
450         return
451     st.divider()
452     gid = st.selectbox("Pilih ID teks untuk detail", list(tampil.index))
453     with st.container(border=True):
454         render_detail(gid, clean_idx)
455 
456 
457 def page_multilabel():
458     st.title("🤌 Multilabel Analysis")
459     st.caption("Distribusi, kombinasi, ko-okurensi, korelasi, dan imbalance label")
460     thr = active_threads()
461     if not thr:
462         st.warning("Pilih minimal satu utas berita pada filter global di sidebar.")
463         return
464     df, Y, sp, tag = build_tables()
465     d = df[df["thread"].isin(thr)]
466 
467     f1, f2 = st.columns([3, 1])
468     fokus = f1.multiselect("Filter: hanya teks yang memuat label", LABELS,
key="ml_label",
469                            placeholder="semua teks (tanpa filter label)")
470     mode = f2.radio("Cocokkan", ["salah satu", "semua"], horizontal=True,
key="ml_mode",
471                     help="salah satu = teks memuat minimal satu label terpilih; semua
= memuat semua label terpilih")
472     if fokus:
473         cocok = all if mode == "semua" else any
474         d = d[d["labels"].map(lambda L: cocok(l in L for l in fokus))]
475     if not len(d):
476         st.info("Tidak ada teks yang cocok dengan filter.")
477         return
```

```
    st.caption(f"Menampilkan **{len(d):,}** teks")
    Yf = Y.loc[d.index]
    n = len(d)

    k1, k2, k3, k4 = st.columns(4)
    k1.metric("Cardinality", f"{d['n_label'].mean():.2f}", "label per teks")
    k2.metric("Density", f"{d['n_label'].mean() / len(LABELS):.3f}")
    k3.metric("Teks multilabel", f"{(d['n_label'] >= 2).mean():.1%}", f"{(d['n_label'] >= 2).sum():,} teks")
    k4.metric("Kombinasi unik", f"{d['labels'].nunique():,}")

    kiri, kanan = st.columns(2)
    dist = Yf.sum().loc[LABELS].sort_values()
    fig = go.Figure(go.Bar(
        x=dist.values, y=dist.index, orientation="h",
        marker_color=[LABEL_COLORS[l] for l in dist.index],
        text=[f"{v} ({v / n:.1%})" for v in dist.values], textposition="outside",
        hovertemplate="%{y}: %{x} teks<extra></extra>",
    ))
    fig.update_xaxes(range=[0, dist.max() * 1.3])
    fig.update_layout(title="Distribusi label")
    kiri.plotly_chart(fig_base(fig), width="stretch")

    per = d["n_label"].value_counts().sort_index()
    fig = go.Figure(go.Bar(
        x=per.index.astype(str) + " label", y=per.values,
        marker_color=WARNA["utama"],
        text=[f"{v} ({v / n:.1%})" for v in per.values], textposition="outside",
        hovertemplate="%{x}: %{y} teks<extra></extra>",
    ))
    fig.update_yaxes(range=[0, per.max() * 1.22])
    fig.update_layout(title="Jumlah label per teks")
    kanan.plotly_chart(fig_base(fig), width="stretch")

    kiri, kanan = st.columns(2)
    komb = d["labels"].map(" + ".join).value_counts().head(12).iloc[::-1]
    fig = go.Figure(go.Bar(
        x=komb.values, y=komb.index, orientation="h",
        marker_color=[LABEL_COLORS[k.split(" + ")[0]] for k in komb.index],
        text=komb.values, textposition="outside",
        hovertemplate="%{y}: %{x} teks<extra></extra>",
    ))
    fig.update_xaxes(range=[0, komb.max() * 1.18])
    fig.update_layout(title="12 kombinasi label terbanyak")
    kiri.plotly_chart(fig_base(fig, 420), width="stretch")

    co = pd.DataFrame(
        [[int((Yf[a] & Yf[b]).sum()) for b in LABELS] for a in LABELS],
        index=LABELS, columns=LABELS,
    )
    skala = kanan.radio("Skala ko-okurensi", ["Jumlah teks", "% dari teks berlabel baris"], horizontal=True)
    if skala == "Jumlah teks":
        z, fmt, judul = co.values, ".0f", "Ko-okurensi label (jumlah teks)"
```

```
    else:
        diag = pd.Series(np.diag(co), index=LABELS).replace(0, np.nan)   # label tanpa teks -> kosong
        z = co.div(diag, axis=0).mul(100).round(1).values
        fmt, judul = ".0f", "Ko-okurensi label (% teks berlabel baris)"
    fig = px.imshow(
        z, x=LABELS, y=LABELS, text_auto=fmt, aspect="auto",
        color_continuous_scale=["#f0efec", "#9ec5f4", "#3987e5", "#184f95"],
    )
    fig.update_layout(title=judul, coloraxis_showscale=False)
    kanan.plotly_chart(fig_base(fig, 420), width="stretch")

    kiri, kanan = st.columns([1, 1], gap="large")
    corr = Yf.corr().round(2)
    fig = px.imshow(
        corr.values, x=LABELS, y=LABELS, text_auto=".2f", zmin=-1, zmax=1, aspect="auto",
        color_continuous_scale=["#2a78d6", "#f0efec", "#e34948"],
    )
    fig.update_layout(title="Korelasi antarlabel (koefisien phi)", coloraxis_showscale=False)
    kiri.plotly_chart(fig_base(fig), width="stretch")

    dist_full = Yf.sum()
    imb = pd.DataFrame({
        "label": dist_full.index,
        "teks": dist_full.values,
        "% teks": (dist_full.values / n * 100).round(1),
        "rasio imbalance": (dist_full.max() / dist_full.replace(0, np.nan)).round(1).values,
    }).set_index("label")
    kanan.dataframe(
        imb, width="stretch",
        column_config={
            "% teks": st.column_config.ProgressColumn("% teks", min_value=0, max_value=100, format="%.1f%%"),
            "rasio imbalance": st.column_config.NumberColumn("rasio imbalance", format="%.1f×"),
        },
    )
    kanan.caption("Rasio imbalance = jumlah teks label terbanyak ÷ jumlah teks label tersebut (kosong = label "
                  "tidak muncul pada filter ini). Tanpa filter, CURSING adalah kelas paling langka.")


def page_ner():
    st.title("🎯 NER Analysis")
    st.caption("Distribusi entitas, struktur tag BIO, contoh span, dan sasaran serangan")
    thr = active_threads()
    if not thr:
        st.warning("Pilih minimal satu utas berita pada filter global di sidebar.")
        return
```

```
    df, Y, sp, tag = build_tables()
    d = df[df["thread"].isin(thr)]
    spf = sp[sp["id"].isin(d.index)]
    n = len(d)

    tab_dist, tab_top, tab_target = st.tabs(["Distribusi", "Top entitas & contoh", "Sasaran serangan"])

    with tab_dist:
        k1, k2, k3, k4 = st.columns(4)
        k1.metric("Total entitas", f"{len(spf):,}")
        k2.metric("Teks dengan entitas", f"{(d['n_entitas'] > 0).mean():.1%}")
        k3.metric("Entitas per teks", f"{(len(spf) / n):.2f}", "rata-rata")
        k4.metric("Token ber-tag O", f"{tag.get('O', 0) / sum(tag.values()):.1%}", f"{sum(tag.values()):,} token total")

        kiri, kanan = st.columns(2)
        ent_n = spf["tipe"].value_counts().reindex(ENT_TYPES).fillna(0).astype(int).sort_values()
        n_teks = {t: d["tipe_entitas"].map(lambda T: t in T).sum() for t in ENT_TYPES}
        fig = go.Figure(go.Bar(
            x=ent_n.values, y=ent_n.index, orientation="h",
            marker_color=[ENTITY_COLORS[t] for t in ent_n.index],
            text=[f"{v} · {n_teks[t] / n:.0%} teks" for t, v in zip(ent_n.index, ent_n.values)],
            textposition="outside",
            hovertemplate="%{y}: %{x} entitas<extra></extra>",
        ))
        fig.update_xaxes(range=[0, ent_n.max() * 1.42])
        fig.update_layout(title="Entitas per tipe")
        kiri.plotly_chart(fig_base(fig), width="stretch")

        per = d["n_entitas"].value_counts().sort_index()
        fig = go.Figure(go.Bar(
            x=per.index.astype(str), y=per.values, marker_color=WARNA["utama"],
            text=[f"{v} ({v / n:.0%})" for v in per.values], textposition="outside",
            hovertemplate="%{x} entitas: %{y} teks<extra></extra>",
        ))
        fig.update_yaxes(range=[0, per.max() * 1.2])
        fig.update_layout(title="Jumlah entitas per teks")
        kanan.plotly_chart(fig_base(fig), width="stretch")

        kiri, kanan = st.columns(2)
        panjang = spf["n_token"].value_counts().sort_index()
        fig = go.Figure(go.Bar(
            x=panjang.index.astype(str) + " token", y=panjang.values, marker_color=WARNA["utama"],
            text=panjang.values, textposition="outside",
            hovertemplate="panjang %{x}: %{y} entitas<extra></extra>",
        ))
        fig.update_yaxes(range=[0, panjang.max() * 1.15])
        fig.update_layout(title="Panjang entitas (token)")
        kiri.plotly_chart(fig_base(fig, 330), width="stretch")

```

```
        tag_labels = ["O", "B-", "I-"]
        tag_values = [tag.get(k, 0) for k in tag_labels]
        fig = go.Figure(go.Pie(
            labels=[f"{k} ({v / sum(tag_values):.1%})" for k, v in zip(tag_labels, tag_values)],
            values=tag_values, hole=0.62, sort=False,
            marker=dict(colors=["#d8d7d1", WARNA["utama"], "#9ec5f4"]),
            textinfo="label", textposition="outside",
        ))
        fig.update_layout(title="Komposisi tag BIO (dominasi tag O)")
        kanan.plotly_chart(fig_base(fig, 330), width="stretch")

    with tab_top:
        kiri, kanan = st.columns([3, 2], gap="large")
        with kiri:
            jml = st.slider("Tampilkan top-N entitas", 5, 30, 15)
            tipe_filter = st.multiselect("Filter tipe entitas", ENT_TYPES, default=ENT_TYPES)
        if tipe_filter:
            kandidat = spf[spf["tipe"].isin(tipe_filter)]
        else:
            kandidat = spf
        if not len(kandidat):
            st.info("Tidak ada entitas untuk tipe terpilih.")
        else:
            grup = kandidat.groupby(["teks", "tipe"]).agg(jumlah=("teks", "size")).reset_index()
            grup = grup.sort_values("jumlah", ascending=False).head(jml).iloc[::-1]
            label_y = grup["teks"] + "  ·  " + grup["tipe"]
            fig = go.Figure(go.Bar(
                x=grup["jumlah"].values, y=label_y, orientation="h",
                marker_color=[ENTITY_COLORS[t] for t in grup["tipe"]],
                text=grup["jumlah"].values, textposition="outside",
                hovertemplate="%{customdata}: %{x} kemunculan<extra></extra>",
                customdata=grup["teks"],
            ))
            fig.update_xaxes(range=[0, grup["jumlah"].max() * 1.15])
            fig.update_layout(title=f"Top {len(grup)} entitas tersering")
            st.plotly_chart(fig_base(fig, 30 + 26 * len(grup)), width="stretch")

            top_daftar = kandidat["teks"].value_counts().head(40).index.tolist()
            pilihan = st.selectbox("Pilih entitas untuk melihat contoh kalimatnya", top_daftar)
            tipe_terpilih = kandidat[kandidat["teks"] == pilihan]["tipe"].value_counts()
            st.markdown(
                "Tipe entitas ini: "
                + " ".join(chip(t, ENTITY_COLORS[t]) + f" ×{v}" for t, v in tipe_terpilih.items()),
                unsafe_allow_html=True,
            )
            contoh_id = kandidat[kandidat["teks"] == pilihan]["id"].drop_duplicates().head(4)
            for cid in contoh_id:
```

```
                g = gold_map()[cid]
                st.markdown(f"**{cid}** · " + label_chips(g["labels"]),
unsafe_allow_html=True)
                st.markdown(
                    highlight_spans(g["text"], g["spans"], emphasize=pilihan),
                    unsafe_allow_html=True,
                )
                st.markdown(" ", unsafe_allow_html=True)

    with tab_target:
        st.caption(
            "Dari teks yang menyandang tiap jenis serangan, berapa persen memuat
entitas dengan tipe tertentu — "
            "memperlihatkan siapa yang jadi sasaran tiap jenis serangan."
        )
        mat = np.zeros((len(ATT_LABELS), len(ENT_TYPES)))
        for i, l in enumerate(ATT_LABELS):
            sub = d[d["labels"].map(lambda L: l in L)]
            for j, t in enumerate(ENT_TYPES):
                if len(sub):
                    mat[i, j] = sub["tipe_entitas"].map(lambda T: t in T).sum() /
len(sub) * 100
        fig = px.imshow(
            mat, x=ENT_TYPES, y=ATT_LABELS, text_auto=".0f", aspect="auto",
            color_continuous_scale=["#f0efec", "#9ec5f4", "#3987e5", "#184f95"],
        )
        fig.update_layout(title="% teks berlabel serangan yang memuat tiap tipe
entitas", coloraxis_showscale=False)
        st.plotly_chart(fig_base(fig, 360), width="stretch")
        st.caption(
            "GOV_ORG (terutama KPAI) muncul di mayoritas teks serangan — sasaran
dominan polemik ini; "
            "PERSON langka karena jarang ada tokoh perorangan yang diserang."
        )


def page_pipeline():
    st.title("🧹 Pipeline Data")
    st.caption("Cleaning, normalisasi, dan seleksi domain — dataset asli hingga gold
dataset")
    clean = load_clean()
    removed = load_support("removed_rows.csv")
    clog = load_support("cleaning_log.csv")
    nlog = load_support("normalization_log.csv")
    sid = load_support("sample_ids.csv")
    gold = load_gold()

    asli = len(clean) + len(removed)
    domain = int(clean["in_domain"].sum())
    pilot = int((sid["bagian"] == "pilot").sum())
    k1, k2, k3, k4, k5 = st.columns(5)
    k1.metric("Dataset asli", f"{asli:,}")
    k2.metric("Setelah cleaning", f"{len(clean):,}", f"-{asli - len(clean)} baris")
```

```
    k3.metric("Corpus domain", f"{domain:,}", f"{len(clean) - domain:,} luar domain (disimpan)")
    k4.metric("Sampel anotasi", f"{(sid['bagian'] == 'utama').sum():,}", f"+{pilot} pilot")
    k5.metric("Gold dataset", f"{len(gold):,}", "1 UNANNOTATABLE dikeluarkan")

    st.subheader("Cleaning")
    st.dataframe(
        clog[["step", "name", "action", "rows_affected"]], width="stretch",
        hide_index=True, height=280,
        column_config={
            "step": st.column_config.TextColumn("langkah", width="small"),
            "name": st.column_config.TextColumn("nama", width="small"),
            "action": st.column_config.TextColumn("aksi", width="large"),
            "rows_affected": st.column_config.NumberColumn("baris terdampak", format="%d"),
        },
    )
    with st.expander("Contoh sebelum → sesudah tiap langkah cleaning"):
        for r in clog.itertuples():
            if r.example_before and r.example_after:
                st.markdown(f"**{r.step} · {r.name}** — {r.action}")
                st.code(r.example_before, language="text")
                st.code(r.example_after, language="text")

    st.subheader("Normalisasi")
    k1, k2, k3 = st.columns(3)
    k1.metric("Kata diganti", f"{int(nlog['jumlah'].sum()):,}")
    k2.metric("Pasangan unik", f"{len(nlog):,}")
    k3.metric("Kamus normalisasi", f"{len(load_support('kamus_normalisasi.csv')):,} entri")

    kiri, kanan = st.columns(2)
    per_jenis = nlog.groupby("jenis")["jumlah"].sum().sort_values()
    fig = go.Figure(go.Bar(
        x=per_jenis.values, y=per_jenis.index, orientation="h",
        marker_color=[JENIS_NORM.get(j, "#888888") for j in per_jenis.index],
        text=per_jenis.values, textposition="outside",
        hovertemplate="%{y}: %{x} kata<extra></extra>",
    ))
    fig.update_xaxes(range=[0, per_jenis.max() * 1.15])
    fig.update_layout(title="Kata yang diganti per jenis normalisasi")
    kiri.plotly_chart(fig_base(fig, 300), width="stretch")

    top = nlog.sort_values("jumlah", ascending=False).head(20).iloc[::-1]
    fig = go.Figure(go.Bar(
        x=top["jumlah"].values, y=(top["asal"] + "  →  " + top["hasil"]).values, orientation="h",
        marker_color=[JENIS_NORM.get(j, "#888888") for j in top["jenis"]],
        text=top["jumlah"].values, textposition="outside",
        hovertemplate="%{y}: %{x} kali<extra></extra>",
    ))
    fig.update_xaxes(range=[0, top["jumlah"].max() * 1.12])
    fig.update_layout(title="20 penggantian tersering")
```

```
    kanan.plotly_chart(fig_base(fig, 520), width="stretch")

    st.markdown("#### Verifikasi istilah proteksi (jumlah sebutan tidak boleh berkurang)")
    c1, c2 = st.columns(2)
    for kata, sel in zip(["kpai", "djarum"], [c1, c2]):
        pola = rf"(?<!\w){kata}(?!\w)"
        a = int(clean["text_clean"].str.lower().str.count(pola).sum())
        b = int(clean["text_norm"].str.lower().str.count(pola).sum())
        sel.metric(f"Sebutan “{kata}”", f"{a} → {b}", f"+{b - a} setelah normalisasi")

    st.subheader("Baris dibuang")
    alasan = removed["reason"].value_counts()
    kiri, kanan = st.columns([1, 3])
    fig = go.Figure(go.Bar(
        x=alasan.values, y=alasan.index, orientation="h", marker_color="#eb6834",
        text=alasan.values, textposition="outside",
        hovertemplate="%{y}: %{x} baris<extra></extra>",
    ))
    fig.update_xaxes(range=[0, alasan.max() * 1.25])
    fig.update_layout(title="Alasan pembuangan")
    kiri.plotly_chart(fig_base(fig, 200), width="stretch")
    kanan.dataframe(
        removed[["id", "text_clean", "orig_label", "reason", "duplicate_of"]],
        width="stretch", hide_index=True, height=220,
        column_config={"text_clean": st.column_config.TextColumn("teks", width="large")},
    )


PAGES = {
    "📊 Overview": page_overview,
    "🔎 Browse & Search": page_browse,
    "🏷️ Multilabel Analysis": page_multilabel,
    "🎯 NER Analysis": page_ner,
    "🧹 Pipeline Data": page_pipeline,
}

if not GOLD_PATH.exists() or not CLEAN_PATH.exists():
    st.error("File dataset tidak ditemukan di folder `dataset/`. Jalankan aplikasi dari folder proyek KelpNLP_UTS.")
    st.stop()

with st.sidebar:
    st.markdown("# 🗂️ KelpNLP")
    st.caption("Dataset Explorer — UTS Pemrosesan Bahasa Alami")
    st.divider()
    halaman = st.radio("Navigasi", list(PAGES.keys()), label_visibility="collapsed", key="nav")
    st.divider()
    st.caption("Filter global")
    st.multiselect(
        "Utas berita", [1, 2, 3], default=[1, 2, 3], key="thread_filter",
        format_func=lambda t: f"Utas {t} — {BERITA[t]}",
```

```
    )
    st.divider()
    if hashlib.sha256(GOLD_PATH.read_bytes()).hexdigest() == SHA_GOLD:
        st.caption("✅ Gold dataset SHA-256 terverifikasi")
    else:
        st.warning("Gold dataset berbeda dari versi rilis notebook.")
    st.caption("Data dibaca read-only dari folder `dataset/`.")

PAGES[halaman]()

```